



Département de génie informatique et génie logiciel

INF2010

Structures de données et algorithmes

Rapport TP3 partie 2

Soumis par

Nicolas Mihali (2435049)

Akhan Mehmet Sozen (2420324)

Groupe 2

8 mars 2026

Polytechnique Montréal

## 1. Expliquez le fonctionnement de votre système (logique de l'implémentation)

Mon système repose sur un arbre AVL qui contient des objets Task.

Comme Task implémente Comparable<Task> en comparant les priorityId, les tâches sont rangées dans l'arbre selon leur priorité. Cela permet d'utiliser les propriétés d'un arbre de recherche binaire pour générer les rapports dans le bon ordre.

La classe TaskManager contient trois parties principales de logique : le déchiffrement des coûts, la génération des rapports de priorité et l'évaluation du coût final.

La méthode prepareData(BinaryNode<Task> node, int parentKey) sert à déchiffrer les coûts des tâches. Le coût d'une tâche est chiffré dans le champ executionCost, et il doit être déchiffré à l'aide de la clé du parent en utilisant l'opérateur XOR.

La logique suivie est la suivante :

- si le nœud est nul, on arrête le traitement;
- on récupère la tâche stockée dans le nœud;
- on déchiffre executionCost avec la clé du parent;
- on marque la tâche comme déchiffrée avec isDecrypted = true;
- on récupère la clé du nœud courant;
- on appelle récursivement la méthode sur le fils gauche et le fils droit en leur transmettant cette clé.

Cette méthode correspond à un parcours préordre, car il faut traiter le parent avant ses enfants. La méthode generateReport(boolean ascending) génère une liste de tâches triées selon leur priorité. Elle crée d'abord une liste vide, puis récupère la racine de l'arbre. Ensuite :

- si ascending vaut true, elle appelle buildAscending;
- sinon, elle appelle buildDescending.

La méthode buildAscending utilise un parcours en ordre :

- enfant de gauche
- nœud
- enfant de droite

Comme l'arbre est un BST basé sur priorityId, ce parcours retourne les tâches de la plus petite à la plus grande priorité. La méthode buildDescending utilise le même principe, mais dans l'ordre inverse. Cela permet d'obtenir les tâches de la plus grande à la plus petite priorité. La méthode evaluateFinalCost(BinaryNode<Task> node) calcule le coût final d'un sous-arbre de tâches. La logique appliquée est la suivante :

- si le nœud est nul, la méthode retourne 0;

- si le nœud possède deux enfants, on calcule récursivement le coût final de l'enfant de gauche et de l'enfant de droite, puis on applique l'opérateur du parent à ces deux résultats;
- sinon, si le nœud est une feuille ou ne possède qu'un seul enfant, il conserve simplement son propre coût déchiffré.

Cette méthode suit un parcours postordre, car le coût d'un parent dépend d'abord des résultats de ses enfants. La méthode `compute(int a, int b, String op)` applique l'opération mathématique demandée entre les valeurs de l'enfant de gauche et de l'enfant de droite.

Les cas gérés sont :

- + pour l'addition
- - pour la soustraction
- \* pour la multiplication
- ^ pour la puissance

Dans le cas de la puissance, j'ai utilisé une boucle qui multiplie `a` par lui-même `b` fois. Si l'opérateur n'est pas valide, la méthode lance une exception.

## **2. Faites l'analyse de la complexité temporelle et spatiale pour la génération de rapports de priorité et pour le calcul du coût total des tâches.**

### **Analyse de `generateReport`**

#### **Complexité temporelle**

La méthode `generateReport` commence par :

- créer une liste vide, ce qui coûte  $O(1)$ ;
- récupérer la racine de l'arbre, ce qui coûte  $O(1)$ ;
- appeler soit `buildAscending`, soit `buildDescending`.

Dans les deux cas, la logique est la même du point de vue de la complexité. Pour chaque appel de `buildAscending`:

- le test `if (node == null)` se fait en  $O(1)$
- l'appel récursif sur le sous-arbre gauche se fait une fois
- l'ajout `list.add(node.value)` se fait en  $O(1)$
- l'appel récursif sur le sous-arbre droit se fait une fois

Chaque nœud de l'arbre est visité exactement une seule fois. Le travail effectué pour chaque nœud est constant. Donc :

- coût par nœud :  $O(1)$
- nombre de nœuds visités :  $n$

On obtient  $O(n \cdot 1) = O(n)$

La méthode `buildDescending` a exactement le même comportement, sauf que l'ordre des appels gauche/droite est inversé. Sa complexité temporelle est donc aussi  $O(n)$

La méthode `generateReport` s'exécute donc  $O(1) + O(n) = O(n)$

## Complexité spatiale

La liste contient toutes les tâches de l'arbre. Elle peut donc contenir jusqu'à  $n$  éléments ce qui donne  $O(n)$ . Les méthodes `buildAscending` et `buildDescending` sont récursives. La profondeur maximale de la récursion correspond à la hauteur de l'arbre que l'on nomme  $h$ .

Comme la structure utilisée est un AVL, l'arbre reste équilibré, donc :

$$h = O(\log n)$$

L'espace total utilisé par la génération du rapport est  $O(n) + O(\log n) = O(n)$

## Analyse de la méthode `evaluateFinalCost`

### Complexité temporelle

La méthode `evaluateFinalCost` traite chaque nœud une seule fois.

Pour un appel sur un nœud donné :

- le test `if (node == null)` coûte  $O(1)$ ;
- l'accès à `node.value` coûte  $O(1)$ ;
- le test sur le nombre d'enfants coûte  $O(1)$ ;
- si le nœud a deux enfants, on fait deux appels récursifs;
- ensuite, on appelle `compute`.

La méthode `compute` effectue :

- pour `+`, `-` et `*` : une opération constante, donc  $O(1)$ ;

- pour  $\wedge$ , une boucle qui s'exécute  $b$  fois.

Dans une analyse simplifiée classique du TP, on considère souvent les opérations arithmétiques comme constantes. Dans ce cas, `compute` est considéré en  $O(1)$ . On peut donc dire que chaque nœud demande un travail constant, et comme tous les nœuds sont visités une fois alors  $O(n)$

## Complexité spatiale

La méthode n'utilise pas de structure comme une liste ou un tableau. L'espace supplémentaire provient essentiellement de la récursion. La profondeur maximale des appels récursifs correspond à la hauteur de l'arbre :  $h = O(\log n)$  car l'arbre est AVL. La complexité spatiale est donc  $O(\log n)$

### 3. Utilisation des SIAG dans ce TP

J'ai utilisé un SIAG comme outil d'aide à la réflexion et à la validation. Il m'a surtout servi à confirmer la logique des parcours d'arbres à utiliser dans chaque méthode.

Par exemple, je m'en suis servi pour vérifier :

- quel parcours convient quand un parent doit être traité avant ses enfants;
- quel parcours permet d'obtenir les éléments d'un BST dans l'ordre croissant;
- quel parcours utiliser lorsqu'un parent dépend du résultat de ses sous-arbres;
- comment structurer l'analyse de complexité de façon claire.